



On utilisera le type Caml suivant pour implanter les *tries* :

```
type trie =  
  { mot_complet : string option; suite : (char * trie) list }
```

Le type prédéfini `type 'a option = None | Some of 'a` sert à représenter une valeur éventuellement absente. Chaque nœud d'un *trie* contient les informations suivantes :

- Si le nœud marque la fin d'un mot *s* (ce qui correspond aux nœuds étoilés de la figure), alors le champ `mot_complet` contient `Some s`, sinon ce champ contient `None`.
- Le champ `suite` contient une liste qui associe les caractères aux nœuds.

**C.1 :** Écrire la valeur Caml de type `trie` correspondant à la figure ci-dessus.

**C.2 :** Écrire une fonction `compte_mots` qui compte le nombre de mots dans un *trie*.

**C.3 :** Écrire une fonction `select` qui prend en argument un *trie* et une lettre, et renvoie le *trie* correspondant aux mots commençant par cette lettre. Si ce *trie* n'existe pas (parce qu'aucun mot ne commence par cette lettre), la fonction devra lancer une exception `Absent`, que l'on définira. On utilisera la fonction prédéfinie `List.assoc` pour effectuer la recherche dans la liste des sous-arbres `suite`.

**C.4 :** Écrire une fonction `recherche` qui vérifie si une chaîne de caractères est un mot dans un *trie* donné. La fonction devra prendre un argument supplémentaire *i*, qui représente la position dans le mot associée au nœud courant. Le *i*<sup>e</sup> caractère d'une chaîne *s* s'obtient en écrivant `s.[i]`, le premier caractère étant numéroté 0. La longueur de la chaîne *s* s'écrit `String.length s`. On emploiera la fonction `select`.

#### Exercice D :

On s'intéresse aux expressions booléennes écrites à l'aide des connecteurs `or` (« ou » booléen), `and` (« et » booléen), `not` (négation booléenne), des constantes `true` et `false`, et de variables. Par exemple :

$$(x \text{ or } y) \text{ and } \text{not}(x \text{ and } y)$$

.

**D.1 :** Définir un type Caml `bool_exp` pour ces expressions.

**D.2 :** Écrire une fonction `eval` permettant d'évaluer de telles expressions. Cette fonction devra prendre en paramètre un environnement associant les variables à leur valeur.

**D.3** : Les connecteurs booléens considérés satisfont en particulier les identités

$$\begin{aligned}not(a \text{ or } b) &= not(a) \text{ and } not(b) \\not(a \text{ and } b) &= not(a) \text{ or } not(b) \\not(not(a)) &= a \\not(true) &= false \\not(false) &= true\end{aligned}$$

Ces identités permettent de transformer toute expression booléenne en une expression équivalente où les négations ne sont appliquées qu'à des variables. Par exemple, l'expression  $not(x \text{ and } (y \text{ or } not(z)))$  peut être transformée en  $not(x) \text{ or } not(y \text{ or } not(z))$  puis en  $not(x) \text{ or } (not(y) \text{ and } not(not(z)))$  et enfin en  $not(x) \text{ or } (not(y) \text{ and } z)$ .

Écrire une fonction Caml **normalise** qui réalise cette transformation.